

CSE-4878

Machine Learning and Data Mining Lab

Guided Laboratory Manual

Classes 1–25: Instructional Laboratories and Project Readiness

Course Information	Details
Course Code	CSE-4878
Course Title	Machine Learning and Data Mining Lab
Credits	1
Laboratory Pattern	15 weeks $\times$ 2 classes per week = 30 classes
Prerequisite	CSE-3636 (Artificial Intelligence Lab)
Co-requisite	CSE-4877 (Machine Learning and Data Mining)
Primary Environment	Python 3.x, Jupyter Notebook or Google Colab
Manual Coverage	Classes 1–25 only
Excluded	Classes 26–30: final project evaluation

Manual purpose: This manual converts the original experiment sequence into a self-guided practice system. Each instructional class contains enough theory, mathematical guidance, implementation steps, checkpoints, independent practice, troubleshooting, and viva preparation for a student to complete the laboratory without depending on continuous instructor intervention.

Source alignment: The supplied course manual explicitly sequences Classes 1–25 as instructional laboratories and project-readiness preparation, while Classes 26–30 are reserved exclusively for final project evaluation. The sequence is Python $\rightarrow$ data analysis $\rightarrow$ preprocessing $\rightarrow$ data mining $\rightarrow$ machine learning $\rightarrow$ text analytics $\rightarrow$ project readiness.

Contents

1. How to Use This Manual
2. Laboratory Environment and Setup
3. Standard Notebook Structure
4. Reproducibility and Data Leakage Rules
5. Guided Practice Classes 1–25
6. Cross-Class Skill Checklist
7. Troubleshooting Guide
8. Final Project Readiness Boundary

1. How to Use This Manual

The manual is designed as a learning-and-practice document rather than a list of experiment titles. For every class, complete the guided practice in order. Do not jump directly to the final task. The progression is deliberate: students first build Python and numerical skills, then learn data handling and statistical reasoning, then preprocessing and data mining, then predictive models, clustering and ensemble learning, and finally text analytics.

- Read the Objective, Learning Outcomes and Theory before executing code.
- Type important code yourself at least once; do not rely only on copy-paste.
- Run each code block immediately after writing it and inspect the output.
- After every major step, compare your output with the Expected Result described in the class.
- Record unexpected results in the notebook instead of silently deleting or rerunning cells until the result looks correct.
- Complete the Guided Practice before attempting the Independent Practice.
- Answer the Viva questions in your own words before looking at the next class.
- Keep the same notebook discipline throughout the semester so that your later project work becomes reproducible.

Minimum notebook sections required in every class

Section	What the student must write
Objective	What the experiment is intended to accomplish.
Learning Outcomes	Specific skills that should be demonstrated.
Theory	Definitions, equations and assumptions relevant to the experiment.
Libraries	Packages imported and their purpose.
Dataset	Source, shape, variables, target and relevant limitations.
Implementation	Ordered code cells with meaningful comments.
Results	Tables, metrics, plots and important outputs.
Interpretation	What the results mean; distinguish observation from explanation.
Limitation / Error Analysis	Unexpected behavior, assumptions, weaknesses and possible causes.
Conclusion	A concise evidence-based conclusion.
Viva Preparation	Answers to the supplied questions.

2. Laboratory Environment and Setup

The supplied manual recommends Python 3.x with Jupyter Notebook or Google Colab and lists NumPy, Pandas, SciPy, Matplotlib, Seaborn, scikit-learn, mlxtend, NLTK and/or spaCy, WordCloud, Git/GitHub and Overleaf as the main environment. `filecite turn0file0 L113-L123`

2.1 Recommended installation

```
python --version
python -m pip install numpy pandas scipy matplotlib seaborn scikit-learn mlxtend nltk spacy
wordcloud jupyter
```

If using Google Colab: Run the installation cell only for packages that are not already available. Restart the runtime only when Colab explicitly requests it.

2.2 Verify the environment

```
import sys
import numpy as np
```

```

import pandas as pd
import scipy
import sklearn
import matplotlib
import seaborn as sns

print("Python:", sys.version)
print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
print("SciPy:", scipy.__version__)
print("scikit-learn:", sklearn.__version__)
print("Matplotlib:", matplotlib.__version__)
print("Seaborn:", sns.__version__)

```

2.3 Recommended folder structure

```

CSE-4878-Lab/
├── data/
├── notebooks/
├── figures/
├── tables/
├── reports/
└── README.md

```

Important: Do not commit very large raw datasets to GitHub unless the dataset license and repository strategy explicitly allow it. The source manual recommends referencing large datasets through a stable source instead of unnecessarily committing them to GitHub. [filecite](#) [turn0file0](#) [L107-L110](#)

3. Standard Notebook Structure

Use the following skeleton at the beginning of every notebook. Replace the placeholder text rather than deleting the structure.

```

# CSE-4878 Machine Learning and Data Mining Lab
# Week __, Class __
# Experiment: _____
# Student: _____
# ID: _____

## 1. Objective
## 2. Learning Outcomes
## 3. Background / Theory
## 4. Required Libraries
## 5. Dataset
## 6. Implementation
## 7. Results and Visualization
## 8. Interpretation
## 9. Limitation / Error Analysis
## 10. Conclusion
## 11. Viva Preparation

```

4. Reproducibility and Data Leakage Rules

The original manual requires correct separation between training and testing data from the preprocessing laboratories onward and requires students to document preprocessing decisions. It also asks students to retain

experiment logs, figures and tables because these become evidence for later project work.

filecite turn0file0 L100-L112

- Set a random seed whenever an algorithm contains randomness and record the value.
- Split the data before fitting preprocessing operations that learn from the data.
- Fit scalers, imputers, vectorizers and similar learned transformations on training data only.
- Never use the test set to choose a model, hyperparameter or preprocessing strategy.
- Keep a model comparison table rather than remembering only the best result.
- Save figures with meaningful filenames and record what each figure demonstrates.
- When an experiment changes one parameter, keep the other major settings fixed whenever possible.
- Report limitations and failures honestly; an unsuccessful experiment is still useful evidence when interpreted correctly.

4.1 A safe supervised-learning pipeline

1. Load raw data
2. Inspect and clean obvious structural problems
3. Separate X and y
4. Split train and test
5. Fit preprocessing on training data
6. Transform training and test data
7. Train model(s) using training data
8. Select/tune using training/validation evidence
9. Evaluate once on the held-out test set
10. Interpret results and limitations

Core principle: The test set is evidence about generalization, not a development workspace.

5. Guided Practice Classes 1–25

The following 25 classes preserve the instructional experiment families and sequence of the supplied manual. The original source provides the purpose, learning objectives, core laboratory procedure, student extension, expected outputs and viva questions for each class; this version expands those elements into guided practice so students can learn by execution rather than by following a short checklist. The final evaluation classes are intentionally omitted. filecite turn0file0 L124-L162

Week 1 – Class 1: Python Fundamentals and Notebook Setup

Purpose: Establish the minimum Python skills required for independent machine-learning laboratory work.

Item	Details
Class	Week 1, Class 1
Topic	Python Fundamentals and Notebook Setup
CLO	CLO1
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week1_Class1_Python_Fundamentals_and_Notebook_Setup.ipynb

Learning Outcomes

- Use variables and common Python data types.
- Manipulate lists, tuples, dictionaries and sets.

- Write conditions, loops and reusable functions.
- Create a structured Jupyter or Colab notebook.
- Read a small CSV file and inspect software versions.

Background and Theory

Python is the programming foundation for the subsequent NumPy, Pandas and scikit-learn laboratories.

A laboratory notebook should be executable from top to bottom and should make the experiment understandable to another reader.

Key Equations / Concepts

- Arithmetic mean: $\bar{x} = (1/n) \sum x_i$
- Range: $\max(x) - \min(x)$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Create a new notebook and write the standard notebook header from this manual.

Step 2. Create integer, floating-point, string and Boolean variables. Use `type()` to inspect each one.

Step 3. Create a list of five marks, a tuple of three course codes, a dictionary containing student information, and a set containing repeated values. Explain what changes after duplicates are inserted into the set.

Step 4. Write an if/elif/else block that converts a numeric mark into a letter grade.

Step 5. Write both a for loop and a while loop that print the values 1 through 10.

Step 6. Write reusable functions named `mean_value`, `maximum_value`, `minimum_value` and `value_range`.

Step 7. Read the small CSV string in the code template and convert it into a list of dictionaries.

Step 8. Record Python and package versions and save the notebook using the required naming convention.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
# Basic Python laboratory
scores = [72, 81, 65, 90, 77]

mean_score = sum(scores) / len(scores)
maximum = max(scores)
minimum = min(scores)
score_range = maximum - minimum

print("Mean:", mean_score)
print("Maximum:", maximum)
print("Minimum:", minimum)
print("Range:", score_range)

def describe(values):
    return {
        "count": len(values),
        "mean": sum(values) / len(values),
        "min": min(values),
        "max": max(values)
    }
```

```
print(describe(scores))

import csv
from io import StringIO

csv_text = "name,score\nA,72\nB,81\nC,65\n"
reader = csv.DictReader(StringIO(csv_text))
rows = list(reader)
print(rows)
```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Create a function that returns mean, median, minimum, maximum and range for a list without using NumPy or Pandas.
2. Create five test cases, including an empty list and a list containing negative numbers. Explain how your function handles edge cases.

Expected Output and Evidence

- A clean notebook with executable cells.
- Working functions and test cases.
- A CSV-reading demonstration.
- Version information.
- A short reflection explaining why functions and reproducibility matter.

Troubleshooting and Self-Diagnosis

- If a function raises `ZeroDivisionError` for an empty list, decide whether to return `None` or raise a meaningful error.
- If indentation errors occur, inspect every line inside the function or loop.
- If a CSV is not parsed as expected, inspect the delimiter and header names.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why are functions useful in ML notebooks?

Your answer: _____

Q. What is the difference between a list and a NumPy array?

Your answer: _____

Q. Why is reproducibility important?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 1 – Class 2: NumPy for Numerical Computing

Purpose: Build the numerical foundation required for statistics, regression, clustering and neural networks.

Item	Details
Class	Week 1, Class 2
Topic	NumPy for Numerical Computing
CLO	CLO1
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week1_Class2_NumPy_for_Numerical_Computing.ipynb

Learning Outcomes

- Create one-, two- and three-dimensional arrays.
- Inspect shape, ndim, size and dtype.
- Perform indexing, slicing and reshaping.
- Use vectorized arithmetic.
- Compute common statistical summaries and compare vectorization with loops.

Background and Theory

NumPy arrays provide compact numerical storage and vectorized operations.

shape describes the size of each dimension; size is the total number of elements; ndim is the number of dimensions; dtype describes the stored numeric type.

Key Equations / Concepts

- Mean: $\mu = (1/n)\sum x_i$
- Population variance: $\sigma^2 = (1/n)\sum (x_i - \mu)^2$
- Standard deviation: $\sigma = \sqrt{\sigma^2}$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Create 1D, 2D and 3D arrays and write down their shapes before printing them.

Step 2. Use indexing to extract one element, one row, one column and a rectangular submatrix.

Step 3. Reshape a sequence of 24 values into shapes 4×6, 6×4 and 2×3×4. Verify that the total size remains 24.

Step 4. Use vectorized arithmetic to standardize an array using $(x - \text{mean})/\text{std}$.

Step 5. Calculate mean, standard deviation, minimum, maximum, sum, argmin and argmax using NumPy.

Step 6. Run the vectorization-versus-loop comparison. Repeat it twice and discuss why the timings can vary.

Step 7. Create one deliberate dtype conversion and inspect the resulting dtype.

Step 8. Record the results in a small table in the notebook.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
import numpy as np
```

```

import time

a = np.array([10, 20, 30, 40, 50])
matrix = np.arange(1, 13).reshape(3, 4)

print("shape:", matrix.shape)
print("ndim:", matrix.ndim)
print("size:", matrix.size)
print("dtype:", matrix.dtype)

print("First row:", matrix[0])
print("Column 2:", matrix[:, 1])
print("Submatrix:", matrix[0:2, 1:3])

print("Mean:", a.mean())
print("Std:", a.std())
print("Min:", a.min())
print("Max:", a.max())
print("Argmin:", a.argmin())
print("Argmax:", a.argmax())

# Vectorization versus loop
x = np.arange(1_000_000)

t0 = time.perf_counter()
y_vectorized = x * 2 + 5
t1 = time.perf_counter()

t2 = time.perf_counter()
y_loop = []
for value in x:
    y_loop.append(value * 2 + 5)
t3 = time.perf_counter()

print("Vectorized time:", t1 - t0)
print("Loop time:", t3 - t2)

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Implement min-max normalization with NumPy only.
2. Write a function that accepts a 2D array and returns row-wise means and column-wise means.
3. Explain one situation where changing dtype can create unexpected results.

Expected Output and Evidence

- Array creation and inspection.
- Correct indexing and reshaping.
- Statistical summaries.
- A timing comparison.
- A short explanation of vectorization.

Troubleshooting and Self-Diagnosis

- A reshape fails when the requested dimensions do not contain the same number of elements.
- If integer division or dtype behavior looks unexpected, inspect dtype explicitly.
- Timing on very small arrays can be noisy; use a larger array for comparison.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why is vectorization useful?

Your answer: _____

Q. What is the difference between shape and size?

Your answer: _____

Q. Why might dtype matter?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 2 – Class 1: Pandas and DataFrames

Purpose: Develop practical tabular-data manipulation skills.

Item	Details
Class	Week 2, Class 1
Topic	Pandas and DataFrames
CLO	CLO1, CLO2
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week2_Class1_Pandas_and_DataFrames.ipynb

Learning Outcomes

- Create Series and DataFrames.
- Load and inspect CSV data.
- Select, filter and sort records.
- Group and summarize data.
- Export a derived table.

Background and Theory

Pandas provides the main tabular-data interface used in data mining and machine-learning preprocessing.

A DataFrame is a labeled two-dimensional table; Series represents a single labeled vector.

Key Equations / Concepts

- Group mean: $\mu_g = (1/ng)\sum x_i$ for observations belonging to group g

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Create the notebook structure and load the embedded CSV dataset.

Step 2. Inspect head, tail, shape, dtypes and descriptive statistics.

Step 3. Use column selection to create a smaller DataFrame.

Step 4. Use loc to filter students with score ≥ 80 and iloc to select the first three rows and first three columns.

Step 5. Sort by score in descending order and inspect the top three observations.

Step 6. Group by department and calculate mean score, mean attendance and number of students.

Step 7. Export the grouped summary to CSV and read it back to verify that the file was written correctly.

Step 8. Write three observations about the dataset without making unsupported causal claims.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
import pandas as pd
from io import StringIO

csv_text = """student,department,semester,score,attendance
A,CSE,3,82,91
B,CSE,3,74,85
```

```

C,EEE,3,88,94
D,CSE,4,91,96
E,EEE,4,69,78
F,CSE,4,77,88
"""

df = pd.read_csv(StringIO(csv_text))

print(df.head())
print(df.shape)
print(df.dtypes)
print(df.describe(numeric_only=True))

print(df[["student", "score"]])
print(df.loc[df["score"] >= 80])
print(df.iloc[:3, :3])

sorted_df = df.sort_values("score", ascending=False)
print(sorted_df)

summary = (
    df.groupby("department")
      .agg(mean_score=("score", "mean"),
           mean_attendance=("attendance", "mean"),
           students=("student", "count"))
      .reset_index()
)

print(summary)
summary.to_csv("department_summary.csv", index=False)

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Add a performance category column: Excellent, Good or Needs Improvement using a rule you define and document.
2. Create a semester-level summary.
3. Write one paragraph explaining why dtype inspection should happen before modeling.

Expected Output and Evidence

- Dataset profile.
- Filtered and sorted tables.
- Grouped summary.
- Exported CSV.
- Evidence-based observations.

Troubleshooting and Self-Diagnosis

- If a numeric column is read as object, inspect unusual characters or missing values.
- Use loc for label-based selection and iloc for position-based selection.

- When exporting, use `index=False` unless the index is intentionally part of the data.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. When should `loc` be used instead of `iloc`?

Your answer: _____

Q. Why inspect dtypes before modeling?

Your answer: _____

Q. What does `groupby` provide?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 2 – Class 2: Data Visualization and Statistical Description

Purpose: Connect statistical summaries to visual evidence.

Item	Details
Class	Week 2, Class 2
Topic	Data Visualization and Statistical Description
CLO	CLO1
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week2_Class2_Data_Visualization_and_Statistical_Description.ipynb

Learning Outcomes

- Compute descriptive statistics.
- Generate histograms, boxplots, bar charts and scatterplots.
- Interpret distributions and outliers.
- Analyze correlation without confusing it with causation.

Background and Theory

Visualization reveals central tendency, spread, skewness, outliers and relationships that numerical summaries alone may hide.

A histogram describes a numerical distribution; a bar chart compares categories; a boxplot summarizes quartiles and potential outliers.

Key Equations / Concepts

- Sample variance: $s^2 = \sum (x_i - \bar{x})^2 / (n-1)$
- Pearson correlation: $r = \text{cov}(X, Y) / (\sigma_X \sigma_Y)$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Create the synthetic dataset and inspect its numerical summaries.

Step 2. Calculate mean, median, mode where meaningful, variance and standard deviation for at least two variables.

Step 3. Create a histogram and explain center, spread and skewness.

Step 4. Create a boxplot and identify possible outliers.

Step 5. Create a categorical frequency plot.

Step 6. Create a scatterplot and describe the visible relationship.

Step 7. Compute and visualize the correlation matrix.

Step 8. Write separate Observation and Interpretation subsections. In Interpretation, state why correlation does not by itself establish causation.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
import pandas as pd
```

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

rng = np.random.default_rng(42)
df = pd.DataFrame({
    "study_hours": rng.normal(5, 1.5, 120).clip(0, 10),
    "sleep_hours": rng.normal(7, 1.0, 120).clip(3, 10),
})
df["score"] = (50 + 5*df["study_hours"] + rng.normal(0, 7, 120)).clip(0, 100)
df["group"] = rng.choice(["A", "B", "C"], size=len(df))

print(df.describe())
print(df[["study_hours", "sleep_hours", "score"]].corr())

plt.figure(figsize=(6,4))
plt.hist(df["score"], bins=12, edgecolor="black")
plt.xlabel("Score")
plt.ylabel("Frequency")
plt.title("Score Distribution")
plt.show()

plt.figure(figsize=(6,4))
plt.boxplot(df["score"], vert=True)
plt.ylabel("Score")
plt.title("Score Boxplot")
plt.show()

plt.figure(figsize=(6,4))
sns.countplot(data=df, x="group")
plt.title("Group Frequency")
plt.show()

plt.figure(figsize=(6,4))
sns.scatterplot(data=df, x="study_hours", y="score")
plt.title("Study Hours versus Score")
plt.show()

plt.figure(figsize=(6,4))
sns.heatmap(df[["study_hours", "sleep_hours", "score"]].corr(), annot=True, fmt=".2f")
plt.title("Correlation Matrix")
plt.show()

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Choose a different random seed and compare the distributions.
2. Create a second variable with an intentionally strong linear relationship and verify that the correlation changes.
3. Write one example where a plot could be misleading because of axis choices or binning.

Expected Output and Evidence

- Descriptive-statistics table.
- At least four complementary plots.
- Correlation matrix.
- Interpretation paragraph.

Troubleshooting and Self-Diagnosis

- A histogram depends on bin width; try two reasonable bin counts and compare.
- A correlation near zero does not necessarily mean no relationship if the relationship is nonlinear.
- Do not label an observation an outlier solely because it is visually distant; explain the rule used.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Does correlation imply causation?

Your answer: _____

Q. What does a boxplot reveal?

Your answer: _____

Q. When is a histogram preferable to a bar chart?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 3 – Class 1: Data Quality and Cleaning

Purpose: Prepare imperfect real-world data for analysis.

Item	Details
Class	Week 3, Class 1
Topic	Data Quality and Cleaning
CLO	CLO2
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week3_Class1_Data_Quality_and_Cleaning.ipynb

Learning Outcomes

- Identify missing values, duplicates and inconsistent categorical values.
- Detect outliers with IQR.
- Apply justified cleaning strategies.
- Document every data-quality decision.

Background and Theory

Cleaning improves reliability, but cleaning is not the same as deleting unusual values. Every modification should have a reason and should preserve meaningful information where possible.

Key Equations / Concepts

- $IQR = Q3 - Q1$
- Lower fence = $Q1 - 1.5 \times IQR$
- Upper fence = $Q3 + 1.5 \times IQR$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Inspect missing-value counts for every column.

Step 2. Count duplicate rows and display the duplicated records.

Step 3. Inspect unique city labels and identify inconsistent capitalization.

Step 4. Standardize the categorical values without changing their meaning.

Step 5. Calculate Q1, Q3, IQR and the $1.5 \times IQR$ fences for age.

Step 6. Inspect the candidate outlier rather than deleting it automatically. Explain why the value may be an entry error or a legitimate observation.

Step 7. Choose appropriate missing-value strategies and document why they were chosen.

Step 8. Compare row count, missing counts and distributions before and after cleaning.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
import pandas as pd
import numpy as np

df = pd.DataFrame({
```

```

    "age": [21, 22, np.nan, 24, 24, 25, 120, 23],
    "city": ["Dhaka", "dhaka", "Dhaka", "Chattogram", "Chattogram", "Dhaka", "Dhaka",
"Dhaka"],
    "score": [75, 81, 79, np.nan, 85, 90, 88, 90]
})

print("Missing values:")
print(df.isna().sum())

print("Duplicate rows:", df.duplicated().sum())

df["city_clean"] = (
    df["city"].astype("string")
    .str.strip()
    .str.lower()
    .str.title()
)

q1 = df["age"].quantile(0.25)
q3 = df["age"].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr

outlier_mask = (df["age"] < lower) | (df["age"] > upper)
print("Outlier rows:")
print(df[outlier_mask])

clean = df.copy()
clean["age"] = clean["age"].fillna(clean["age"].median())
clean["score"] = clean["score"].fillna(clean["score"].median())
clean = clean.drop_duplicates()
print(clean)

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Create a data-quality decision table with columns: Issue, Evidence, Action, Reason, Possible Risk.
2. Repeat the experiment using mean imputation and compare it with median imputation.
3. Create a second dataset where removing an outlier would actually destroy useful information and explain why.

Expected Output and Evidence

- Missing-value table.
- Duplicate report.
- Inconsistent-category report.
- Outlier evidence.
- Cleaned dataset and decision log.

Troubleshooting and Self-Diagnosis

- Never replace all missing values with zero without establishing that zero is semantically valid.
- If an outlier is removed, preserve the original dataset and save a cleaned copy.
- If a categorical value has multiple spellings, inspect all unique values before standardizing.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why should missing values not always be replaced by zero?

Your answer: _____

Q. What can happen if outliers are removed without justification?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 3 – Class 2: Data Transformation, Normalization and Encoding

Purpose: Transform heterogeneous attributes into model-ready features while preventing leakage.

Item	Details
Class	Week 3, Class 2
Topic	Data Transformation, Normalization and Encoding
CLO	CLO2
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week3_Class2_Data_Transformation_Normalization_and_Encoding.ipynb

Learning Outcomes

- Apply standardization and min-max normalization.
- Encode categorical variables.
- Perform a correct train/test split.
- Explain why preprocessing must be fitted on training data only.
- Demonstrate the effect of scaling on distance calculations.

Background and Theory

Standardization centers and scales a feature using its training mean and standard deviation. Min-max normalization maps values into a chosen interval. One-hot encoding represents categorical levels numerically. Scaling is particularly important for distance-based methods and margin-based models.

Key Equations / Concepts

- Standard score $z = (x - \mu_{\text{train}}) / \sigma_{\text{train}}$
- Min-max: $x' = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Separate features and target before modeling.

Step 2. Split the data into training and testing subsets using a fixed random_state.

Step 3. Identify numerical and categorical columns.

Step 4. Fit StandardScaler only on the training subset.

Step 5. Transform both training and test subsets using the already-fitted transformer.

Step 6. Apply one-hot encoding to the categorical feature.

Step 7. Inspect the resulting matrix shape and explain why the number of columns changed.

Step 8. Demonstrate the leakage error by fitting a scaler on the full dataset, then explain why the result is invalid.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler, OneHotEncoder
```

```

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

df = pd.DataFrame({
    "age": [20, 21, 22, 23, 24, 25, 26, 27, 28, 29],
    "income": [20, 22, 25, 27, 30, 34, 38, 42, 50, 60],
    "city": ["A", "A", "B", "B", "C", "C", "A", "B", "C", "A"],
    "target": [0, 0, 0, 1, 1, 1, 1, 1, 1, 1]
})

X = df.drop(columns="target")
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)

numeric = ["age", "income"]
categorical = ["city"]

preprocess = ColumnTransformer([
    ("num", StandardScaler(), numeric),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical)
])

X_train_t = preprocess.fit_transform(X_train)
X_test_t = preprocess.transform(X_test)

print("Train shape:", X_train_t.shape)
print("Test shape:", X_test_t.shape)

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Build a reusable ColumnTransformer + Pipeline that combines preprocessing and a LogisticRegression model.
2. Compare StandardScaler and MinMaxScaler on the same numerical features.
3. Explain which of Decision Tree, SVM and KNN is most sensitive to scaling and why.

Expected Output and Evidence

- Correct train/test split.
- Preprocessed feature matrix.
- Before/after range comparison.
- Leakage explanation.
- Reusable pipeline.

Troubleshooting and Self-Diagnosis

- If stratification fails, inspect whether every class has enough samples.
- If OneHotEncoder creates an unexpected number of columns, inspect unique category values.

- Never call `fit_transform` on the test set.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. What is data leakage?

Your answer: _____

Q. Why must a scaler be fitted on training data?

Your answer: _____

Q. Which algorithms are especially sensitive to feature scale?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 4 – Class 1: Similarity and Dissimilarity Measures

Purpose: Introduce distance and similarity measures used in KNN, clustering and vector comparison.

Item	Details
Class	Week 4, Class 1
Topic	Similarity and Dissimilarity Measures
CLO	CLO1, CLO3
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week4_Class1_Similarity_and_Dissimilarity_Measures.ipynb

Learning Outcomes

- Calculate Euclidean and Manhattan distance.
- Calculate cosine similarity.
- Construct a pairwise distance matrix.
- Relate the choice of measure to the data representation.

Background and Theory

Distance measures quantify dissimilarity; similarity measures quantify closeness. The same observations can appear more or less similar depending on the chosen metric and feature scaling.

Key Equations / Concepts

- Euclidean $d(x,y)=\sqrt{\sum(x_i-y_i)^2}$
- Manhattan $d(x,y)=\sum|x_i-y_i|$
- Cosine $\text{sim}(x,y)=x \cdot y / (\|x\| \|y\|)$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Write the Euclidean-distance calculation for A and B manually on paper before using Python.

Step 2. Verify the manual result with SciPy.

Step 3. Calculate Manhattan distance for the same pair.

Step 4. Calculate cosine similarity and explain why vectors with the same direction can have high cosine similarity even when their magnitudes differ.

Step 5. Construct a pairwise distance matrix for three observations.

Step 6. Multiply one feature by 100 and repeat Euclidean distance. Explain what changed.

Step 7. Standardize the features and repeat the comparison.

Step 8. Relate the results to KNN and text-vector comparison.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
import numpy as np
from scipy.spatial.distance import euclidean, cityblock, cosine
from sklearn.metrics import pairwise_distances
```

```

A = np.array([1, 2, 3])
B = np.array([2, 4, 6])
C = np.array([3, 2, 1])

print("A-B Euclidean:", euclidean(A, B))
print("A-B Manhattan:", cityblock(A, B))
print("A-B Cosine similarity:", 1 - cosine(A, B))

X = np.array([A, B, C])
D_euclidean = pairwise_distances(X, metric="euclidean")
D_manhattan = pairwise_distances(X, metric="manhattan")
D_cosine = pairwise_distances(X, metric="cosine")

print("Euclidean distance matrix:\n", D_euclidean)
print("Manhattan distance matrix:\n", D_manhattan)
print("Cosine distance matrix:\n", D_cosine)

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Construct two pairs of vectors where Euclidean and cosine give different notions of closeness.
2. Use a small TF-IDF matrix and compare cosine similarity between documents.
3. Write a comparison table with Euclidean, Manhattan and cosine: formula, scale sensitivity, typical use.

Expected Output and Evidence

- Manual calculation.
- Verified distances.
- Similarity matrix.
- Scaling comparison.
- Metric-selection explanation.

Troubleshooting and Self-Diagnosis

- Cosine similarity is undefined for a zero vector; check vector norms.
- Do not confuse cosine distance with cosine similarity: distance is commonly $1 - \text{similarity}$.
- Scaling can change Euclidean distances dramatically.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why is cosine similarity useful for text vectors?

Your answer: _____

Q. How does scaling change Euclidean distance?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 4 – Class 2: EDA and Statistical Analysis Laboratory

Purpose: Integrate statistical and visualization techniques into a complete evidence-based EDA workflow.

Item	Details
Class	Week 4, Class 2
Topic	EDA and Statistical Analysis Laboratory
CLO	CLO1, CLO2
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week4_Class2_EDA_and_Statistical_Analysis_Laboratory.ipynb

Learning Outcomes

- Conduct structured EDA on a new dataset.
- Analyze missingness and class distribution.
- Generate appropriate univariate and bivariate plots.
- Translate EDA findings into preprocessing and modeling considerations.

Background and Theory

EDA is not merely plotting. It is the process of understanding the dataset sufficiently to justify later preprocessing and modeling choices.

Key Equations / Concepts

- Class proportion = class count / total count
- Missing rate = missing count / total rows $\times 100\%$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Load the dataset and identify its rows, columns, target and class labels.

Step 2. Create a data dictionary with variable name, type and role.

Step 3. Calculate missingness percentages and class proportions.

Step 4. Identify at least three numerical features for distribution analysis.

Step 5. Create class-distribution, histogram, boxplot and correlation visualizations.

Step 6. Inspect whether strong feature correlations may affect later modeling or interpretation.

Step 7. Write at least five observations and convert each observation into a possible modeling implication.

Step 8. Finish with a one-page EDA summary that another student could use to plan preprocessing.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer(as_frame=True)
df = data.frame.copy()
```

```

df["target_name"] = df["target"].map(dict(enumerate(data.target_names)))

print(df.shape)
print(df.dtypes.value_counts())
print(df.isna().sum().sort_values(ascending=False).head())
print(df["target_name"].value_counts())
print(df.describe().T.head())

fig = plt.figure(figsize=(7,4))
sns.countplot(data=df, x="target_name")
plt.title("Class Distribution")
plt.show()

fig = plt.figure(figsize=(7,4))
sns.histplot(data=df, x="mean radius", hue="target_name", kde=True)
plt.title("Feature Distribution")
plt.show()

fig = plt.figure(figsize=(7,4))
sns.boxplot(data=df, x="target_name", y="mean radius")
plt.title("Group-wise Feature Distribution")
plt.show()

corr = df.select_dtypes("number").corr()
fig = plt.figure(figsize=(10,8))
sns.heatmap(corr, cmap="Greys")
plt.title("Correlation Matrix")
plt.show()

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Repeat the EDA using another built-in classification dataset.
2. Identify one potential data-quality concern and propose a validation step.
3. Create an EDA checklist that you can reuse for your final text project.

Expected Output and Evidence

- Complete EDA notebook.
- Dataset summary.
- Visualizations.
- Findings and modeling implications.

Troubleshooting and Self-Diagnosis

- Do not plot every feature automatically without interpretation; select plots that answer questions.
- High correlation does not automatically mean a feature must be removed.
- For categorical targets, inspect counts before relying on accuracy.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Which EDA finding would cause you to change preprocessing?

Your answer: _____

Q. How can class imbalance appear visually?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 5 – Class 1: Apriori: Frequent Itemset Mining

Purpose: Implement association-rule mining using the Apriori principle.

Item	Details
Class	Week 5, Class 1
Topic	Apriori: Frequent Itemset Mining
CLO	CLO3
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week5_Class1_Apriori_Frequent_Itemset_Mining.ipynb

Learning Outcomes

- Explain support, confidence and lift.
- Generate frequent itemsets.
- Generate and filter association rules.
- Interpret useful and misleading rules.
- Verify a small support calculation manually.

Background and Theory

Apriori uses the downward-closure principle: if an itemset is infrequent, every larger itemset containing it is also infrequent. This reduces the search space but can still require repeated candidate generation.

Key Equations / Concepts

- $\text{Support}(X) = \text{count}(X)/N$
- $\text{Confidence}(X \rightarrow Y) = \text{Support}(X \cup Y) / \text{Support}(X)$
- $\text{Lift}(X \rightarrow Y) = \text{Confidence}(X \rightarrow Y) / \text{Support}(Y)$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Represent each transaction as a list of items.

Step 2. For one itemset, count how many transactions contain it and manually calculate support.

Step 3. Convert transactions into one-hot form using TransactionEncoder.

Step 4. Run Apriori with a chosen minimum support and inspect the resulting itemsets.

Step 5. Generate association rules using a minimum confidence.

Step 6. Sort rules by lift and inspect the strongest rules.

Step 7. Take one high-confidence rule and determine whether lift supports the same interpretation.

Step 8. Write an interpretation that avoids claiming causation from association.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules
```

```

transactions = [
    ["bread", "milk"],
    ["bread", "diaper", "beer", "eggs"],
    ["milk", "diaper", "beer", "cola"],
    ["bread", "milk", "diaper", "beer"],
    ["bread", "milk", "diaper", "cola"],
]

te = TransactionEncoder()
encoded = te.fit(transactions).transform(transactions)
basket = pd.DataFrame(encoded, columns=te.columns_)

freq = apriori(basket, min_support=0.40, use_colnames=True)
rules = association_rules(freq, metric="confidence", min_threshold=0.60)
rules = rules.sort_values(["lift", "confidence"], ascending=False)

print("Frequent itemsets:")
print(freq)
print("\nRules:")
print(rules[["antecedents", "consequents", "support", "confidence", "lift"]])

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Implement support counting for 1-itemsets and one 2-itemset from scratch.
2. Change minimum support and observe how the number of itemsets changes.
3. Construct one example where confidence is high but lift is close to 1.

Expected Output and Evidence

- Frequent-itemset table.
- Association-rule table.
- Manual support calculation.
- Interpretation of top rules.

Troubleshooting and Self-Diagnosis

- If no itemsets appear, lower min_support gradually and explain the choice.
- If rules are empty, lower confidence only after verifying that frequent itemsets exist.
- Do not interpret association as causation.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why can high confidence be misleading?

Your answer: _____

Q. What does lift greater than 1 suggest?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 5 – Class 2: FP-Growth: Frequent Pattern Mining

Purpose: Study FP-Growth as an efficient alternative to Apriori and compare their computational logic.

Item	Details
Class	Week 5, Class 2
Topic	FP-Growth: Frequent Pattern Mining
CLO	CLO3, CLO4
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week5_Class2_FP_Growth_Frequent_Pattern_Mining.ipynb

Learning Outcomes

- Explain the FP-tree concept.
- Mine frequent patterns with FP-Growth.
- Generate association rules.
- Compare Apriori and FP-Growth in terms of candidate generation and runtime.

Background and Theory

FP-Growth compresses transactions into an FP-tree and mines patterns without repeatedly generating the same candidate sets. It addresses one important computational limitation of Apriori.

Key Equations / Concepts

- Use the same support, confidence and lift definitions as in Apriori.

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Use the same transaction dataset from the Apriori class so the comparison is controlled.

Step 2. Run Apriori and record itemset count and runtime.

Step 3. Run FP-Growth with the same minimum support and record itemset count and runtime.

Step 4. Verify whether both algorithms discover equivalent frequent itemsets.

Step 5. Generate rules from the FP-Growth result.

Step 6. Explain the FP-tree idea using a small hand-drawn tree for three transactions.

Step 7. Create a comparison table: candidate generation, compressed representation, itemset count, runtime.

Step 8. Write a conclusion that does not claim FP-Growth is universally faster from one tiny dataset.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
import time
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, fpgrowth, association_rules

transactions = [
    ["bread", "milk"],
    ["bread", "diaper", "beer", "eggs"],
```

```

["milk","diaper","beer","cola"],
["bread","milk","diaper","beer"],
["bread","milk","diaper","cola"],
["bread","beer"],
["milk","diaper"],
["bread","milk","beer"],
]

te = TransactionEncoder()
basket = pd.DataFrame(
    te.fit(transactions).transform(transactions),
    columns=te.columns_
)

t0 = time.perf_counter()
a = apriori(basket, min_support=0.25, use_colnames=True)
t1 = time.perf_counter()

t2 = time.perf_counter()
f = fpgrowth(basket, min_support=0.25, use_colnames=True)
t3 = time.perf_counter()

print("Apriori itemsets:", len(a), "time:", t1-t0)
print("FP-Growth itemsets:", len(f), "time:", t3-t2)

rules_f = association_rules(f, metric="confidence", min_threshold=0.60)
print(rules_f[["antecedents","consequents","support","confidence","lift"]])

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Increase the number of transactions by repeating the transaction patterns and compare runtime cautiously.
2. Explain why small datasets may not reveal the computational advantage clearly.
3. List two situations where Apriori may still be useful pedagogically or practically.

Expected Output and Evidence

- FP-Growth itemsets.
- Rules.
- Apriori versus FP-Growth comparison.
- Interpretation of computational behavior.

Troubleshooting and Self-Diagnosis

- Use identical support thresholds for a fair comparison.
- Do not compare runtimes from different hardware or after one algorithm has warmed up without noting the limitation.
- Equivalent frequent itemsets are more important than tiny runtime differences on small data.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. What problem of Apriori does FP-Growth address?

Your answer: _____

Q. What information is stored in an FP-tree?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 6 – Class 1: Simple Linear Regression

Purpose: Implement and evaluate a basic predictive regression model.

Item	Details
Class	Week 6, Class 1
Topic	Simple Linear Regression
CLO	CLO3
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week6_Class1_Simple_Linear_Regression.ipynb

Learning Outcomes

- Explain the simple linear regression equation.
- Fit a model with one predictor.
- Interpret slope and intercept.
- Calculate MAE, MSE and R^2 .
- Inspect residuals.

Background and Theory

Simple linear regression estimates a continuous target from one predictor. The fitted line minimizes the sum of squared residuals under ordinary least squares.

Key Equations / Concepts

- $\hat{y} = \beta_0 + \beta_1 x$
- Residual $e_i = y_i - \hat{y}_i$
- $MAE = (1/n) \sum |y_i - \hat{y}_i|$
- $MSE = (1/n) \sum (y_i - \hat{y}_i)^2$
- $R^2 = 1 - SSE/SST$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Inspect the synthetic predictor-target relationship with a scatterplot.

Step 2. Fit a LinearRegression model and print intercept and slope.

Step 3. Write the fitted equation in mathematical notation.

Step 4. Calculate predictions and residuals.

Step 5. Compute MAE, MSE and R^2 .

Step 6. Plot the fitted line over the observations.

Step 7. Plot residuals against predictions and inspect whether residuals show obvious structure.

Step 8. Explain whether the relationship appears suitable for linear prediction.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
import numpy as np
import matplotlib.pyplot as plt
```

```

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

rng = np.random.default_rng(42)
X = np.arange(1, 31).reshape(-1, 1)
y = 10 + 2.5 * X.ravel() + rng.normal(0, 5, len(X))

model = LinearRegression()
model.fit(X, y)
pred = model.predict(X)

print("Intercept:", model.intercept_)
print("Slope:", model.coef_[0])
print("MAE:", mean_absolute_error(y, pred))
print("MSE:", mean_squared_error(y, pred))
print("R2:", r2_score(y, pred))

residuals = y - pred

plt.figure(figsize=(7,4))
plt.scatter(X, y, label="Observed")
plt.plot(X, pred, label="Regression line")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()

plt.figure(figsize=(7,4))
plt.axhline(0, linestyle="--")
plt.scatter(pred, residuals)
plt.xlabel("Predicted")
plt.ylabel("Residual")
plt.title("Residual Plot")
plt.show()

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Implement slope and intercept from the ordinary least-squares formulas for a small dataset and compare with scikit-learn.
2. Add one extreme observation and observe how the fitted line changes.
3. Explain why high R^2 does not prove causation.

Expected Output and Evidence

- Regression equation.
- Prediction plot.
- Residual plot.
- MAE, MSE and R^2 .
- Interpretation of fit.

Troubleshooting and Self-Diagnosis

- Do not compare MAE and MSE as if they were in the same units.
- R^2 can be negative on held-out data; interpret it relative to the chosen baseline.
- Residual structure may indicate that a straight line is insufficient.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. What does R^2 measure?

Your answer: _____

Q. Why is a residual important?

Your answer: _____

Q. Can high R^2 prove causation?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 6 – Class 2: Multiple Linear Regression

Purpose: Extend regression to multiple predictors and evaluate coefficient behavior.

Item	Details
Class	Week 6, Class 2
Topic	Multiple Linear Regression
CLO	CLO3
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week6_Class2_Multiple_Linear_Regression.ipynb

Learning Outcomes

- Fit multiple linear regression.
- Interpret coefficients conditional on other predictors.
- Compare simple and multiple models.
- Inspect feature correlations and discuss multicollinearity.

Background and Theory

Multiple regression models a continuous target using several predictors. A coefficient represents the expected target change associated with one predictor while holding the other included predictors constant, under the model assumptions.

Key Equations / Concepts

- $\hat{y} = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$
- VIF concept: large values indicate strong linear dependence among predictors.

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Identify one simple predictor and a set of multiple predictors.

Step 2. Split the dataset into training and testing subsets.

Step 3. Fit a simple regression model and a multiple regression model.

Step 4. Evaluate both models with MAE, MSE and R^2 on the same test set.

Step 5. Create a coefficient table for the multiple model.

Step 6. Compute the feature correlation matrix.

Step 7. Identify any pair of predictors that could create coefficient instability.

Step 8. Write a model-selection conclusion based on test evidence rather than model complexity.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.DataFrame({
```

```

    "study_hours": [2,3,4,5,6,7,8,9,10,11,12,13],
    "attendance": [70,72,75,78,80,82,85,88,90,92,94,96],
    "sleep": [6,7,6,7,8,7,8,7,8,8,7,8],
    "score": [55,60,64,68,73,76,80,84,87,90,94,96]
})

X = df[["study_hours", "attendance", "sleep"]]
y = df["score"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

simple = LinearRegression().fit(X_train[["study_hours"]], y_train)
multiple = LinearRegression().fit(X_train, y_train)

for name, model, Xt in [
    ("Simple", simple, X_test[["study_hours"]]),
    ("Multiple", multiple, X_test)
]:
    pred = model.predict(Xt)
    print(name)
    print("MAE:", mean_absolute_error(y_test, pred))
    print("MSE:", mean_squared_error(y_test, pred))
    print("R2:", r2_score(y_test, pred))
    print()

print("Feature correlations:")
print(X.corr())
print("Multiple coefficients:", dict(zip(X.columns, multiple.coef_)))

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Remove one predictor and compare performance.
2. Create two strongly correlated predictors and observe how coefficient interpretation becomes less stable.
3. Explain why adding a feature can improve training fit but hurt generalization.

Expected Output and Evidence

- Two regression models.
- Metric comparison.
- Coefficient table.
- Correlation analysis.
- Discussion of multicollinearity.

Troubleshooting and Self-Diagnosis

- A coefficient is conditional on the other predictors in the model.
- With very small datasets, test metrics can vary substantially with the split.
- Do not interpret correlation as proof that one variable causes another.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why can adding features hurt generalization?

Your answer: _____

Q. What is multicollinearity?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 7 – Class 1: Decision Tree Classification

Purpose: Understand and implement a tree-based classifier using entropy and information gain.

Item	Details
Class	Week 7, Class 1
Topic	Decision Tree Classification
CLO	CLO3
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week7_Class1_Decision_Tree_Classification.ipynb

Learning Outcomes

- Explain entropy and information gain.
- Train a Decision Tree classifier.
- Visualize a tree.
- Evaluate predictions.
- Compare shallow and deep trees to observe overfitting.

Background and Theory

Decision Trees recursively split the feature space. Entropy measures class uncertainty. Information gain measures the reduction in uncertainty after a split.

Key Equations / Concepts

- Entropy $H(S) = -\sum p_i \log_2(p_i)$
- Information Gain = $H(\text{parent}) - \sum(\text{weighted child entropy})$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Create a tiny two-class example and calculate entropy manually.

Step 2. Load the classification dataset and perform a stratified train/test split.

Step 3. Train trees with depth 2, depth 5 and unlimited depth.

Step 4. Compare training and test performance to identify overfitting.

Step 5. Generate a confusion matrix and classification report.

Step 6. Visualize a shallow tree and inspect the first few decisions.

Step 7. Explain why the first split is selected by the tree.

Step 8. Write a short conclusion about the trade-off between depth and generalization.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import numpy as np
```

```

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

for depth in [2, 5, None]:
    model = DecisionTreeClassifier(
        criterion="entropy", max_depth=depth, random_state=42
    )
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print("Depth:", depth)
    print(confusion_matrix(y_test, pred))
    print(classification_report(y_test, pred, target_names=data.target_names))

model = DecisionTreeClassifier(
    criterion="entropy", max_depth=3, random_state=42
)
model.fit(X_train, y_train)

plt.figure(figsize=(14, 7))
plot_tree(model, filled=False, feature_names=data.feature_names,
          class_names=data.target_names, max_depth=3)
plt.show()

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Change criterion from entropy to gini and compare results.
2. Limit min_samples_leaf and inspect how the tree structure changes.
3. Identify one case where a tree is attractive because interpretability matters.

Expected Output and Evidence

- Manual entropy example.
- Tree visualization.
- Confusion matrix.
- Classification metrics.
- Depth comparison.

Troubleshooting and Self-Diagnosis

- A deep tree can memorize training observations.
- Do not infer importance from a single tree without considering the instability of tree-based importance.
- Use stratification when class distribution matters.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why can a very deep tree overfit?

Your answer: _____

Q. What is information gain?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 7 – Class 2: Support Vector Machine

Purpose: Implement SVM classification and examine margins, scaling and kernels.

Item	Details
Class	Week 7, Class 2
Topic	Support Vector Machine
CLO	CLO3, CLO4
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week7_Class2_Support_Vector_Machine.ipynb

Learning Outcomes

- Explain the separating hyperplane and margin.
- Train linear and RBF-kernel SVMs.
- Understand the conceptual role of C and gamma.
- Evaluate the effect of feature scaling.

Background and Theory

SVM seeks a decision boundary with a large margin. Kernel functions allow the model to represent nonlinear decision boundaries in the original feature space.

Key Equations / Concepts

- Linear decision function: $f(x)=w \cdot x+b$
- Soft-margin objective balances margin size and classification violations through C.

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Train a linear SVM inside a pipeline with StandardScaler.

Step 2. Train an RBF SVM with C=1.

Step 3. Change C to 10 while keeping other settings fixed.

Step 4. Compare precision, recall and F1 rather than only accuracy.

Step 5. Repeat the experiment without scaling and observe the difference.

Step 6. Explain C conceptually as a trade-off between margin flexibility and training violations.

Step 7. Explain gamma conceptually as controlling the locality of the RBF influence.

Step 8. Create a parameter-performance table and justify the configuration you would select for this dataset.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

```

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

configs = [
    ("Linear", "linear", 1, "scale"),
    ("RBF-C1", "rbf", 1, "scale"),
    ("RBF-C10", "rbf", 10, "scale"),
]

for name, kernel, C, gamma in configs:
    pipe = Pipeline([
        ("scale", StandardScaler()),
        ("svc", SVC(kernel=kernel, C=C, gamma=gamma))
    ])
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_test)
    print(name, {
        "accuracy": accuracy_score(y_test, pred),
        "precision": precision_score(y_test, pred),
        "recall": recall_score(y_test, pred),
        "f1": f1_score(y_test, pred)
    })

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Try C values 0.1, 1, 10 and 100.
2. Try two gamma values and plot performance.
3. For a two-feature dataset, visualize the decision boundary for a linear and RBF SVM.

Expected Output and Evidence

- Model comparison.
- Metrics.
- Scaling comparison.
- Parameter-performance table.
- Model-selection rationale.

Troubleshooting and Self-Diagnosis

- SVM is often highly sensitive to scale.
- Do not tune C and gamma on the test set.
- Very large C or extreme gamma can produce unstable or overfit boundaries.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why is scaling important for SVM?

Your answer: _____

Q. What does C control conceptually?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 8 – Class 1: K-Nearest Neighbors

Purpose: Apply distance-based classification and investigate the role of k.

Item	Details
Class	Week 8, Class 1
Topic	K-Nearest Neighbors
CLO	CLO3
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week8_Class1_K_Nearest_Neighbors.ipynb

Learning Outcomes

- Explain nearest-neighbor classification.
- Train KNN for multiple k values.
- Demonstrate the role of scaling.
- Select k using validation evidence.
- Explain underfitting and overfitting as k changes.

Background and Theory

KNN predicts from nearby training observations. Because distance determines neighborhood membership, feature scaling is central to the method.

Key Equations / Concepts

- For k neighbors, prediction is commonly the majority class.
- Euclidean distance: $d(x,y)=\sqrt{\sum(x_i-y_i)^2}$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Split the dataset using stratification.

Step 2. Create a KNN pipeline with StandardScaler.

Step 3. Train KNN for k=1 through k=20.

Step 4. Record the chosen evaluation metric for each k.

Step 5. Plot performance against k.

Step 6. Inspect k=1 as a high-variance setting and a large k as a smoother setting.

Step 7. Compare scaled and unscaled KNN.

Step 8. Choose k using evidence and state why the selection is not simply a guess.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

```

import matplotlib.pyplot as plt

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

k_values = range(1, 21)
scores = []

for k in k_values:
    model = Pipeline([
        ("scale", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    scores.append(accuracy_score(y_test, pred))

best_k = list(k_values)[scores.index(max(scores))]
print("Best k on this held-out comparison:", best_k)

plt.figure(figsize=(7,4))
plt.plot(list(k_values), scores, marker="o")
plt.xlabel("k")
plt.ylabel("Accuracy")
plt.title("KNN Performance versus k")
plt.show()

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Use a validation split instead of choosing k directly from the final test set.
2. Compare Manhattan and Euclidean distance.
3. Create a two-dimensional synthetic dataset and visualize the KNN decision boundary for k=1 and k=15.

Expected Output and Evidence

- k-performance plot.
- Best-k rationale.
- Confusion matrix.
- Metrics.
- Scaling comparison.

Troubleshooting and Self-Diagnosis

- Choosing k directly from the final test set is a form of test-set overuse; use validation evidence for a rigorous workflow.
- KNN can become computationally expensive with large training sets.

- Always inspect feature scales before distance-based modeling.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. What happens when $k=1$?

Your answer: _____

Q. Why does KNN require scaling?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 8 – Class 2: Naive Bayes Classification

Purpose: Implement probabilistic classification and connect it to text-oriented problems.

Item	Details
Class	Week 8, Class 2
Topic	Naive Bayes Classification
CLO	CLO3
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week8_Class2_Naive_Bayes_Classification.ipynb

Learning Outcomes

- Explain Bayes theorem.
- Understand the conditional-independence assumption.
- Train a Naive Bayes classifier.
- Generate probabilities.
- Connect Naive Bayes to sparse text classification.

Background and Theory

Naive Bayes estimates class probabilities using Bayes theorem and assumes conditional independence between features given the class. The assumption is strong but can work well in high-dimensional sparse representations.

Key Equations / Concepts

- $P(C|X) = P(X|C)P(C)/P(X)$
- For multinomial text models, class evidence is based on feature counts or their transformed representation.

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Write Bayes theorem for one class and explain each term.

Step 2. Inspect the text samples and identify words that may be predictive.

Step 3. Split the text data into training and testing subsets.

Step 4. Fit CountVectorizer on training text only.

Step 5. Train MultinomialNB.

Step 6. Generate predictions and class probabilities.

Step 7. Inspect the confusion matrix and classification report.

Step 8. Explain why conditional independence is unrealistic for natural language but still useful as a simplifying assumption.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
```

```

from sklearn.metrics import classification_report, confusion_matrix

texts = [
    "excellent product and fast delivery",
    "great quality and excellent service",
    "very happy with the purchase",
    "good product and helpful support",
    "terrible product and slow delivery",
    "bad quality and poor service",
    "very unhappy with the purchase",
    "poor product and unhelpful support",
    "excellent service and good quality",
    "bad product with terrible support",
]
labels = [1,1,1,1,0,0,0,0,1,0]

X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.30, random_state=42, stratify=labels
)

vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

model = MultinomialNB()
model.fit(X_train_vec, y_train)

pred = model.predict(X_test_vec)
prob = model.predict_proba(X_test_vec)

print("Vocabulary:", vectorizer.get_feature_names_out())
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
print("Probabilities:", prob)

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Create five additional labeled sentences and test the model.
2. Compare CountVectorizer with TF-IDF features.
3. Compare Naive Bayes with one classifier from a previous class using the same text representation.

Expected Output and Evidence

- Probability outputs.
- Metrics.
- Confusion matrix.
- Text-feature representation.
- Comparison paragraph.

Troubleshooting and Self-Diagnosis

- Fit the vectorizer only on training text to avoid vocabulary leakage.
- Very small datasets can produce unstable results; treat the exercise as a method demonstration rather than a research benchmark.
- Inspect the actual tokens if predictions are surprising.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. What is the naive assumption?

Your answer: _____

Q. Why can Naive Bayes work well for sparse text features?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 9 – Class 1: Artificial Neural Network

Purpose: Introduce a practical multilayer neural-network workflow.

Item	Details
Class	Week 9, Class 1
Topic	Artificial Neural Network
CLO	CLO3
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week9_Class1_Artificial_Neural_Network.ipynb

Learning Outcomes

- Explain neurons, weights and activation functions.
- Describe forward propagation and loss.
- Train an MLP classifier.
- Record training and validation performance.
- Recognize overfitting and the role of regularization.

Background and Theory

A neural network applies weighted transformations followed by nonlinear activations. Training changes weights to reduce a loss function. An MLP can model nonlinear relationships but can overfit.

Key Equations / Concepts

- Neuron: $z = w^T x + b$
- Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$
- ReLU: $\max(0, z)$
- Binary cross-entropy: $-[y \log(p) + (1 - y) \log(1 - p)]$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Sketch a single neuron and label input, weight, bias, activation and output.

Step 2. Explain the forward-pass calculation $z = w^T x + b$ for one toy input.

Step 3. Scale the classification features before training the MLP.

Step 4. Train a small MLP and record the architecture.

Step 5. Inspect the loss curve.

Step 6. Evaluate on the held-out test set.

Step 7. Change one architecture parameter, such as hidden-layer width, and compare performance.

Step 8. Explain signs of overfitting and why early stopping can help.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = Pipeline([
    ("scale", StandardScaler()),
    ("mlp", MLPClassifier(
        hidden_layer_sizes=(64, 32),
        activation="relu",
        solver="adam",
        learning_rate_init=0.001,
        max_iter=300,
        early_stopping=True,
        random_state=42
    ))
])

model.fit(X_train, y_train)
pred = model.predict(X_test)

print(classification_report(y_test, pred))

mlp = model.named_steps["mlp"]
plt.figure(figsize=(7,4))
plt.plot(mlp.loss_curve_)
plt.xlabel("Iteration")
plt.ylabel("Loss")
plt.title("MLP Training Loss")
plt.show()

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Compare hidden layers (32,) and (64,32).
2. Change learning_rate_init and observe convergence.
3. Explain why nonlinear activation functions are necessary for learning nonlinear decision boundaries.

Expected Output and Evidence

- Network configuration.
- Learning curve.
- Test metrics.
- Architecture comparison.

- Overfitting discussion.

Troubleshooting and Self-Diagnosis

- MLP training is sensitive to feature scale.
- If convergence warnings appear, inspect scaling, max_iter and learning rate.
- Do not judge a neural network from one random seed alone in a research setting.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why are nonlinear activations needed?

Your answer: _____

Q. What is overfitting in a neural network?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 9 – Class 2: Neural Network Implementation and Comparison

Purpose: Evaluate neural networks against classical models rather than assuming greater complexity means better performance.

Item	Details
Class	Week 9, Class 2
Topic	Neural Network Implementation and Comparison
CLO	CLO3, CLO4
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week9_Class2_Neural_Network_Implementation_and_Comparison.ipynb

Learning Outcomes

- Tune two small MLP configurations.
- Train a classical baseline.
- Compare accuracy, precision, recall and F1.
- Inspect learning behavior and errors.
- Select a model using evidence and computational considerations.

Background and Theory

Model selection depends on data characteristics and empirical evidence. A more complex model is not automatically better. A simpler model may be preferable when it is competitive, easier to explain or cheaper to run.

Key Equations / Concepts

- Generalization gap = training score – validation/test score

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Use one classical model as the baseline.

Step 2. Train two MLP architectures while keeping the data split and evaluation procedure identical.

Step 3. Record training time and test metrics for all models.

Step 4. Inspect the learning curves of the MLP models.

Step 5. Compare false positives and false negatives using confusion matrices.

Step 6. Create one model-comparison table with accuracy, precision, recall, F1 and computational notes.

Step 7. Identify whether the more complex MLP provides a meaningful improvement.

Step 8. Write a model-selection conclusion that considers performance, complexity and limitations.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```

from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

models = {
    "Logistic Regression": Pipeline([
        ("scale", StandardScaler()),
        ("model", LogisticRegression(max_iter=1000, random_state=42))
    ]),
    "MLP-32": Pipeline([
        ("scale", StandardScaler()),
        ("model", MLPClassifier(hidden_layer_sizes=(32,), max_iter=300,
                                early_stopping=True, random_state=42))
    ]),
    "MLP-64-32": Pipeline([
        ("scale", StandardScaler()),
        ("model", MLPClassifier(hidden_layer_sizes=(64,32), max_iter=300,
                                early_stopping=True, random_state=42))
    ])
}

results = []
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    results.append([
        name,
        accuracy_score(y_test, pred),
        precision_score(y_test, pred),
        recall_score(y_test, pred),
        f1_score(y_test, pred)
    ])

for row in results:
    print(row)

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Repeat the comparison using a different random seed and discuss stability.
2. Identify one error made by the best model and inspect the corresponding features.
3. Write a paragraph explaining why 'higher accuracy' alone is not a sufficient model-selection rule.

Expected Output and Evidence

- Comparison table.

- Learning curves.
- Error examples.
- Model-selection rationale.
- Computational comparison.

Troubleshooting and Self-Diagnosis

- Keep preprocessing identical across models.
- Use the same held-out test set for final comparison.
- A small performance difference may not justify a much more complex model.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why might a simpler model be preferable?

Your answer: _____

Q. How do you distinguish training and validation performance?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 10 – Class 1: K-Means Clustering

Purpose: Implement centroid-based clustering and evaluate cluster structure.

Item	Details
Class	Week 10, Class 1
Topic	K-Means Clustering
CLO	CLO3
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week10_Class1_K_Means_Clustering.ipynb

Learning Outcomes

- Explain the K-Means objective.
- Run K-Means for multiple k values.
- Use inertia and elbow analysis.
- Visualize clusters.
- Interpret cluster characteristics and initialization sensitivity.

Background and Theory

K-Means alternates between assigning each point to the nearest centroid and updating centroids to minimize within-cluster squared distance.

Key Equations / Concepts

- Inertia = $\sum_i \min_k \|x_i - \mu_k\|^2$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Generate the synthetic dataset and plot it without labels.

Step 2. Scale the features and explain why scaling can matter.

Step 3. Run K-Means for k=2 through k=8 and record inertia.

Step 4. Plot the elbow curve and identify a defensible candidate k.

Step 5. Run the final K-Means model and visualize cluster labels.

Step 6. Inspect the centroid coordinates.

Step 7. Change random_state and compare cluster assignments.

Step 8. Write a cluster interpretation based on feature values rather than only cluster numbers.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

X, _ = make_blobs(
    n_samples=500, centers=4, cluster_std=1.2, random_state=42
```

```

)
X = StandardScaler().fit_transform(X)

inertias = []
for k in range(2, 9):
    km = KMeans(n_clusters=k, n_init=20, random_state=42)
    km.fit(X)
    inertias.append(km.inertia_)

plt.figure(figsize=(7,4))
plt.plot(range(2,9), inertias, marker="o")
plt.xlabel("k")
plt.ylabel("Inertia")
plt.title("K-Means Elbow Curve")
plt.show()

model = KMeans(n_clusters=4, n_init=20, random_state=42)
labels = model.fit_predict(X)

plt.figure(figsize=(7,5))
plt.scatter(X[:,0], X[:,1], c=labels, s=15)
plt.scatter(model.cluster_centers_[:,0], model.cluster_centers_[:,1],
            marker="X", s=120)
plt.title("K-Means Clusters")
plt.show()

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Use a dataset with a different number of natural groups.
2. Compare `n_init` values and discuss initialization stability.
3. Calculate silhouette score for several `k` values and compare it with the elbow argument.

Expected Output and Evidence

- Cluster assignments.
- Elbow plot.
- Cluster visualization.
- Centroid interpretation.
- Evidence-based `k` selection.

Troubleshooting and Self-Diagnosis

- Inertia always decreases as `k` increases, so an elbow is a heuristic rather than a proof.
- Cluster labels are arbitrary identifiers; cluster 0 in one run is not necessarily the same semantic group as cluster 0 in another.
- Scale features when units differ substantially.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why does K-Means need k ?

Your answer: _____

Q. What does inertia measure?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 10 – Class 2: K-Medoids Clustering

Purpose: Compare medoid-based clustering with K-Means and study robustness to outliers.

Item	Details
Class	Week 10, Class 2
Topic	K-Medoids Clustering
CLO	CLO3, CLO4
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week10_Class2_K_Medoids_Clustering.ipynb

Learning Outcomes

- Explain medoids.
- Run K-Medoids.
- Inspect representative observations.
- Compare K-Medoids with K-Means.
- Discuss robustness and interpretability.

Background and Theory

A medoid is an actual observation selected as a representative of a cluster. Unlike a centroid, it must correspond to a real data point, which can make cluster representatives easier to inspect.

Key Equations / Concepts

- K-Medoids minimizes total dissimilarity between observations and their assigned medoids.

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Install scikit-learn-extra if your environment does not already contain KMedoids.

Step 2. Generate and scale the dataset.

Step 3. Run K-Medoids for $k=3$.

Step 4. Inspect the medoid indices and print the actual observations represented by them.

Step 5. Run K-Means with the same k and compare the representative points.

Step 6. Add a few extreme observations and repeat both methods.

Step 7. Discuss which method appears more robust in the altered dataset.

Step 8. Create a comparison table covering representative type, outlier sensitivity, interpretability and computational considerations.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
# K-Medoids may require scikit-learn-extra.
# Install if needed:
# pip install scikit-learn-extra

import numpy as np
import matplotlib.pyplot as plt
```

```

from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler

try:
    from sklearn_extra.cluster import KMedoids
except ImportError:
    KMedoids = None

X, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.5, random_state=42)
X = StandardScaler().fit_transform(X)

if KMedoids is not None:
    model = KMedoids(n_clusters=3, random_state=42, method="alternate")
    labels = model.fit_predict(X)
    print("Medoid indices:", model.medoid_indices_)
    print("Medoid observations:\n", X[model.medoid_indices_])

    plt.figure(figsize=(7,5))
    plt.scatter(X[:,0], X[:,1], c=labels, s=15)
    plt.scatter(model.cluster_centers_[0], model.cluster_centers_[1],
                marker="x", s=120)
    plt.title("K-Medoids")
    plt.show()
else:
    print("Install scikit-learn-extra to execute K-Medoids.")

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Try $k=2$, 3 and 4 and compare the resulting cluster structure.
2. Construct a dataset with one extreme outlier and explain how the medoid and centroid can differ.
3. Identify a practical problem where real observations as representatives would be useful.

Expected Output and Evidence

- Medoids.
- Cluster comparison.
- Outlier experiment.
- Interpretation.
- K-Means versus K-Medoids table.

Troubleshooting and Self-Diagnosis

- K-Medoids may require an additional package.
- Do not compare cluster labels directly; compare structure and representative observations.
- Small synthetic datasets can exaggerate differences.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. What makes a medoid different from a centroid?

Your answer: _____

Q. Why can medoids be more interpretable?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 11 – Class 1: DBSCAN and Density-Based Clustering

Purpose: Use density-based clustering to identify arbitrary-shaped clusters and noise.

Item	Details
Class	Week 11, Class 1
Topic	DBSCAN and Density-Based Clustering
CLO	CLO3, CLO4
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week11_Class1_DBSCAN_and_Density_Based_Clustering.ipynb

Learning Outcomes

- Explain eps and min_samples.
- Identify core, border and noise points.
- Run DBSCAN.
- Compare density-based clusters with K-Means.
- Analyze parameter sensitivity.

Background and Theory

DBSCAN defines clusters through local density. It does not require the number of clusters in advance and can label isolated observations as noise.

Key Equations / Concepts

- A core point has at least min_samples observations within eps distance under the chosen metric.

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Generate the two-moons dataset and visualize it without labels.

Step 2. Scale the features.

Step 3. Run DBSCAN with three eps values while keeping min_samples fixed.

Step 4. Record number of clusters and number of noise points for each setting.

Step 5. Identify the effect of increasing eps.

Step 6. Change min_samples and inspect how core/noise structure changes.

Step 7. Run K-Means on the same dataset and compare the shapes discovered.

Step 8. Write a parameter-sensitivity conclusion.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN, KMeans

X, _ = make_moons(n_samples=500, noise=0.08, random_state=42)
```

```

X = StandardScaler().fit_transform(X)

for eps in [0.15, 0.25, 0.40]:
    model = DBSCAN(eps=eps, min_samples=5)
    labels = model.fit_predict(X)

    print("eps:", eps, "clusters:", len(set(labels)) - (1 if -1 in labels else 0),
          "noise:", np.sum(labels == -1))

    plt.figure(figsize=(6,4))
    plt.scatter(X[:,0], X[:,1], c=labels, s=12)
    plt.title(f"DBSCAN eps={eps}")
    plt.show()

km = KMeans(n_clusters=2, random_state=42, n_init=20)
km_labels = km.fit_predict(X)

plt.figure(figsize=(6,4))
plt.scatter(X[:,0], X[:,1], c=km_labels, s=12)
plt.title("K-Means on the same data")
plt.show()

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Find a parameter combination that produces too many noise points and explain why.
2. Find a combination that merges distinct groups and explain why.
3. Explain why DBSCAN is attractive when cluster count is unknown or noise is meaningful.

Expected Output and Evidence

- Cluster plots.
- Noise identification.
- Parameter comparison.
- K-Means comparison.
- Interpretation.

Troubleshooting and Self-Diagnosis

- DBSCAN is highly sensitive to scale and parameter choices.
- eps is measured in the feature space after preprocessing.
- Do not assume the visually pleasing clustering is automatically the correct one; define an evaluation or domain criterion.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. What is a core point?

Your answer: _____

Q. Why can DBSCAN discover noise?

Your answer: _____

Q. Why is parameter selection difficult?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 11 – Class 2: Random Forest and Ensemble Learning

Purpose: Understand ensemble learning and compare Random Forest with a single Decision Tree.

Item	Details
Class	Week 11, Class 2
Topic	Random Forest and Ensemble Learning
CLO	CLO3, CLO4
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week11_Class2_Random_Forest_and_Ensemble_Learning.ipynb

Learning Outcomes

- Explain bagging and random feature selection.
- Train a Random Forest.
- Inspect feature importance.
- Compare Random Forest with a Decision Tree.
- Study the effect of the number of trees.

Background and Theory

Random Forest combines multiple decision trees trained with bootstrap samples and randomized feature subsets. The ensemble can reduce variance and improve stability relative to a single tree.

Key Equations / Concepts

- Bagging prediction for classification is commonly majority voting across trees.

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Train a single Decision Tree baseline.

Step 2. Train Random Forest models with different numbers of trees.

Step 3. Compare accuracy, precision, recall and F1.

Step 4. Inspect whether increasing the number of trees changes performance meaningfully.

Step 5. Extract feature importances from the largest forest.

Step 6. Discuss why an ensemble can reduce variance.

Step 7. Discuss why the ensemble can be less directly interpretable than one tree.

Step 8. Write a defensible conclusion explaining whether the ensemble actually improved the evidence.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd
import matplotlib.pyplot as plt
```

```

data = load_breast_cancer()
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

models = {
    "Decision Tree": DecisionTreeClassifier(max_depth=5, random_state=42),
    "RF-50": RandomForestClassifier(n_estimators=50, random_state=42),
    "RF-200": RandomForestClassifier(n_estimators=200, random_state=42)
}

rows = []
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    rows.append({
        "model": name,
        "accuracy": accuracy_score(y_test, pred),
        "precision": precision_score(y_test, pred),
        "recall": recall_score(y_test, pred),
        "f1": f1_score(y_test, pred)
    })

results = pd.DataFrame(rows)
print(results)

rf = models["RF-200"]
importance = pd.Series(rf.feature_importances_, index=data.feature_names)
importance = importance.sort_values(ascending=False).head(10)

plt.figure(figsize=(8,5))
importance.sort_values().plot(kind="barh")
plt.xlabel("Importance")
plt.title("Top Random Forest Feature Importances")
plt.show()

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Change max_depth in the Decision Tree baseline.
2. Change max_features in Random Forest and compare results.
3. Repeat the experiment with a different seed and discuss stability.

Expected Output and Evidence

- Metric comparison.
- Feature-importance plot.
- Parameter comparison.
- Evidence-based explanation of ensemble behavior.

Troubleshooting and Self-Diagnosis

- Feature importance is not a causal explanation.
- A larger number of trees does not guarantee a large performance improvement.
- Keep the data split constant while comparing model settings.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why does bagging help?

Your answer: _____

Q. Why can Random Forest be less interpretable than one tree?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 12 – Class 1: Model Evaluation and Comparison

Purpose: Build rigorous evaluation skills for responsible model comparison.

Item	Details
Class	Week 12, Class 1
Topic	Model Evaluation and Comparison
CLO	CLO3
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week12_Class1_Model_Evaluation_and_Comparison.ipynb

Learning Outcomes

- Construct confusion matrices.
- Calculate accuracy, precision, recall and F1.
- Plot ROC curves and calculate AUC for binary classification.
- Perform stratified cross-validation.
- Create an evidence-based model comparison table.

Background and Theory

Evaluation must match the task and data distribution. Accuracy can be misleading under class imbalance. Precision emphasizes false-positive control; recall emphasizes false-negative control; F1 balances precision and recall.

Key Equations / Concepts

- $\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$
- $\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$
- $\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$
- $\text{F1} = 2\text{PR} / (\text{P} + \text{R})$
- $\text{Specificity} = \text{TN} / (\text{TN} + \text{FP})$

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Train at least two classifiers on the same train/test split.

Step 2. Generate a confusion matrix for each model.

Step 3. Calculate accuracy, precision, recall and F1 manually from the confusion matrix for one model.

Step 4. Plot ROC curves using predicted probabilities.

Step 5. Calculate ROC-AUC.

Step 6. Perform stratified 5-fold cross-validation using the appropriate scoring metric.

Step 7. Create a single model-comparison table.

Step 8. Repeat the analysis conceptually under class imbalance and explain why accuracy may become less informative.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, roc_curve, roc_auc_score
)
import matplotlib.pyplot as plt

data = load_breast_cancer()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

models = {
    "Logistic Regression": Pipeline([
        ("scale", StandardScaler()),
        ("model", LogisticRegression(max_iter=1000, random_state=42))
    ]),
    "SVM": Pipeline([
        ("scale", StandardScaler()),
        ("model", SVC(probability=True, random_state=42))
    ])
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    prob = model.predict_proba(X_test)[:,1]

    print(name)
    print("Confusion matrix:\n", confusion_matrix(y_test, pred))
    print("Accuracy:", accuracy_score(y_test, pred))
    print("Precision:", precision_score(y_test, pred))
    print("Recall:", recall_score(y_test, pred))
    print("F1:", f1_score(y_test, pred))
    print("ROC-AUC:", roc_auc_score(y_test, prob))

    fpr, tpr, _ = roc_curve(y_test, prob)
    plt.plot(fpr, tpr, label=name)

plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves")
plt.legend()
plt.show()

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(models["Logistic Regression"], X, y, cv=cv, scoring="roc_auc")
print("5-fold ROC-AUC:", scores)
print("Mean:", scores.mean(), "Std:", scores.std())
```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Create a deliberately imbalanced binary dataset and compare accuracy with precision, recall and F1.
2. Explain when ROC-AUC is useful and when precision-recall analysis may be more informative.
3. Write a model-selection statement that cites at least three pieces of evidence.

Expected Output and Evidence

- Evaluation table.
- Confusion matrices.
- ROC-AUC.
- Cross-validation results.
- Model-selection statement.

Troubleshooting and Self-Diagnosis

- Use `predict_proba` or `decision_function` appropriately for ROC curves.
- Do not compute cross-validation on a dataset after using the test set for model selection.
- For multiclass tasks, choose the correct averaging strategy and explain it.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why can accuracy be misleading?

Your answer: _____

Q. What does recall emphasize?

Your answer: _____

Q. When is ROC-AUC useful?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.

- Notebook filename follows the manual convention.

Week 12 – Class 2: Text Data EDA and Preprocessing

Purpose: Introduce the text-analysis workflow required for the text-focused final project.

Item	Details
Class	Week 12, Class 2
Topic	Text Data EDA and Preprocessing
CLO	CLO1, CLO2
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week12_Class2_Text_Data_EDA_and_Preprocessing.ipynb

Learning Outcomes

- Load text data into Pandas.
- Analyze text length and class balance.
- Clean text consistently.
- Tokenize and remove stopwords.
- Apply stemming or lemmatization.
- Compare raw and cleaned text.

Background and Theory

Text preprocessing transforms raw language into a consistent representation while attempting to preserve information relevant to the task. The source manual identifies cleaning, tokenization, stopword removal and stemming/lemmatization as core operations for the project workflow.

Key Equations / Concepts

- Document length = number of tokens or characters under the chosen definition.
- Class proportion = class count / total labeled samples.

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Create a DataFrame with text and target columns.

Step 2. Inspect missing values, duplicate texts and class distribution.

Step 3. Calculate character length and word-count distributions.

Step 4. Create a frequency visualization or word cloud.

Step 5. Convert text to lowercase.

Step 6. Remove punctuation and unwanted characters using a documented rule.

Step 7. Tokenize the cleaned text.

Step 8. Remove stopwords using NLTK or another approved tool.

Step 9. Apply stemming or lemmatization and compare the result with the raw text.

Step 10. Select at least five raw/cleaned examples and explain what changed.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```

import re
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter

texts = [
    "The product is excellent and delivery was very fast!",
    "Excellent support; I am very happy with this service.",
    "The product was poor, broken, and arrived late.",
    "Very bad service. The support team did not help.",
    "Good quality and fast response from the company.",
    "Terrible quality and extremely slow delivery."
]
labels = [1,1,0,0,1,0]

df = pd.DataFrame({"text": texts, "label": labels})

df["char_length"] = df["text"].str.len()
df["word_length"] = df["text"].str.split().str.len()

def clean_text(text):
    text = text.lower()
    text = re.sub(r"[^a-z\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df["clean_text"] = df["text"].apply(clean_text)

print(df[["text", "clean_text", "char_length", "word_length"]])
print(df["label"].value_counts())

plt.figure(figsize=(7,4))
sns.histplot(data=df, x="word_length", hue="label", bins=6, multiple="stack")
plt.title("Text Length Distribution")
plt.show()

```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Use NLTK stopwords and compare with a custom stopword list.
2. Compare stemming and lemmatization on at least ten words.
3. Identify words that should not be removed because they carry task-specific meaning.
4. Create a preprocessing function that can be reused in the final project.

Expected Output and Evidence

- Text EDA plots.
- Class-balance table.
- Before/after cleaning examples.
- Cleaned text column.

- Documented preprocessing pipeline.

Troubleshooting and Self-Diagnosis

- Download NLTK resources only when needed.
- Do not remove every short word automatically; some short words can carry meaning.
- Preserve the raw text column so that transformations remain auditable.
- If a language is not English, use language-appropriate tokenization and stopwords resources rather than applying English assumptions.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why should preprocessing be applied consistently?

Your answer: _____

Q. Can removing stopwords ever remove useful information?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

Week 13 – Class 1: Text Vectorization and Final Project Readiness

Purpose: Convert cleaned text into numerical features and verify readiness for the final project without performing final evaluation.

Item	Details
Class	Week 13, Class 1
Topic	Text Vectorization and Final Project Readiness
CLO	CLO2, CLO3, CLO5
Suggested laboratory time	50 minutes guided work + independent practice
Notebook submission	Week13_Class1_Text_Vectorization_and_Final_Project_Readiness.ipynb

Learning Outcomes

- Build TF-IDF features.
- Inspect vocabulary and matrix dimensions.
- Measure sparsity.
- Create a simple baseline.
- Train a quick validation model.
- Prepare an evidence checklist for the project.

Background and Theory

TF-IDF represents terms according to their importance within documents relative to the corpus. It commonly produces a sparse matrix, which is suitable for many classical text-classification algorithms.

Key Equations / Concepts

- $TF\text{-}IDF(t,d) = TF(t,d) \times IDF(t)$
- $IDF(t) = \log((1+n)/(1+df(t))) + 1$ in a common smoothed formulation

Guided Practice: Step-by-Step

Complete the following steps in order. Each checkpoint is intended to confirm that you understand the output before moving forward.

Step 1. Start from the cleaned text representation created in Class 24.

Step 2. Split text into training and testing subsets before fitting TF-IDF.

Step 3. Fit `TfidfVectorizer` on training text only.

Step 4. Inspect vocabulary size, sample vocabulary entries, matrix dimensions and number of non-zero values.

Step 5. Calculate and interpret matrix sparsity.

Step 6. Create a simple baseline classifier using the TF-IDF representation.

Step 7. Record accuracy, precision, recall and F1 where applicable.

Step 8. Create a project evidence table containing: problem, dataset, preprocessing, vectorization, baseline, candidate algorithms, evaluation evidence, ethics, repository and report.

Step 9. Verify that the notebook can be restarted and executed from the first cell to the last without manual intervention.

Starter Implementation

Run the following implementation as a starting point. After confirming that it works, modify at least one parameter or input and observe the effect.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, confusion_matrix

texts = [
    "excellent product and fast delivery",
    "great quality and excellent service",
    "very happy with the purchase",
    "good product and helpful support",
    "terrible product and slow delivery",
    "bad quality and poor service",
    "very unhappy with the purchase",
    "poor product and unhelpful support",
    "excellent service and good quality",
    "bad product with terrible support",
]
labels = [1,1,1,1,0,0,0,0,1,0]

X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.30, random_state=42, stratify=labels
)

pipe = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1,2), min_df=1)),
    ("model", LogisticRegression(max_iter=1000, random_state=42))
])

pipe.fit(X_train, y_train)
pred = pipe.predict(X_test)

tfidf = pipe.named_steps["tfidf"]
X_train_tfidf = tfidf.transform(X_train)

print("Vocabulary size:", len(tfidf.vocabulary_))
print("Matrix shape:", X_train_tfidf.shape)
print("Non-zero values:", X_train_tfidf.nnz)
print("Sparsity:", 1 - X_train_tfidf.nnz /
      (X_train_tfidf.shape[0] * X_train_tfidf.shape[1]))

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))
```

Checkpoint Questions

- What did the output prove at this stage?
- Which assumption or preprocessing decision was made?
- What would happen if the input changed?
- What evidence should be saved from this step for a later report?

Independent Practice

1. Compare unigram TF-IDF with unigram+bigram TF-IDF.

2. Compare TF-IDF with CountVectorizer.
3. Train one additional classical classifier and create a preliminary comparison.
4. Identify three research questions that could be investigated beyond simply maximizing accuracy.

Expected Output and Evidence

- TF-IDF matrix.
- Vocabulary inspection.
- Sparsity measurement.
- Baseline result.
- Project readiness evidence checklist.

Troubleshooting and Self-Diagnosis

- If the vocabulary is unexpectedly small, inspect cleaning and min_df.
- If a test-only word is absent from the training vocabulary, that is expected and is one reason the vectorizer must be fitted only on training data.
- Do not treat a tiny practice dataset as evidence for a publishable model; this class is a workflow rehearsal.

Reflection

Write 4–6 sentences answering: What did I learn? What result surprised me? Which assumption mattered most? What would I change in a larger or noisier dataset?

Viva Preparation

Q. Why is TF-IDF suitable for sparse text?

Your answer: _____

Q. What is a baseline?

Your answer: _____

Q. What evidence should be retained for later project work?

Your answer: _____

Submission Checklist

- Notebook runs from top to bottom without unexplained errors.
- Objective and learning outcomes are completed.
- Theory and equations relevant to the experiment are included.
- Dataset and preprocessing decisions are documented.
- Results and figures have meaningful labels.
- Interpretation is separated from raw observations.
- At least one limitation or error-analysis point is recorded.
- Independent practice is completed.
- Viva questions have been answered in the notebook.
- Notebook filename follows the manual convention.

6. Cross-Class Skill Checklist

Use this checklist after every two weeks. A student should not mark a skill complete merely because code executed; the skill is complete only when the student can explain the result without reading the notebook.

Skill Area	Minimum Competency	Where Practiced
Python	Variables, data structures, conditions, loops, functions, file handling	Classes 1
Numerical computing	Arrays, indexing, reshaping, vectorization, statistics	Class 2
Tabular data	Load, inspect, filter, sort, group, export	Class 3
Statistical reasoning	Mean, median, spread, distributions, correlation	Classes 4 and 8
Data quality	Missing values, duplicates, inconsistent categories, outliers	Class 5
Preprocessing	Scaling, encoding, train/test separation, leakage prevention	Class 6
Distance methods	Euclidean, Manhattan, cosine, pairwise matrices	Class 7
Association mining	Support, confidence, lift, Apriori, FP-Growth	Classes 9 and 10
Regression	Simple and multiple regression, residuals, MAE, MSE, R^2	Classes 11 and 12
Classification	Decision Tree, SVM, KNN, Naive Bayes	Classes 13–16
Neural networks	MLP, activation, loss, training behavior, comparison	Classes 17 and 18
Clustering	K-Means, K-Medoids, DBSCAN	Classes 19–21
Ensembles	Bagging, Random Forest, feature importance	Class 22
Evaluation	Confusion matrix, precision, recall, F1, ROC-AUC, cross-validation	Class 23
Text analytics	Text EDA, cleaning, tokenization, stopwords, stemming/lemmatization	Class 24
Text representation	TF-IDF, vocabulary, sparsity, baseline	Class 25
Research practice	Experiment logs, reproducibility, limitations, evidence tables	Throughout

7. Troubleshooting Guide

Problem	Self-diagnosis procedure
ModuleNotFoundError	Read the package name in the error, install that package in the active environment, then restart the kernel only if necessary.
Kernel runs but output is missing	Check whether the cell was executed, whether the final expression is displayed, and whether an exception occurred earlier.
Shape mismatch	Print <code>X.shape</code> , <code>y.shape</code> and intermediate matrix shapes after every transformation. Confirm that rows still correspond to observations.
Unexpected object/string dtype	Inspect <code>df.dtypes</code> and unique values. Clean formatting problems before numerical operations.
NaN appears after preprocessing	Check the source of missing values and inspect each transformation one at a time.
Model performs extremely well	Check for target leakage, duplicate records across train/test, preprocessing fitted on all data, or features that encode the target.
Model performs extremely poorly	Inspect labels, class balance, preprocessing, feature scale, train/test split and whether the chosen algorithm fits the data representation.
Results change every run	Set <code>random_state</code> where supported and record all stochastic settings. Some algorithms may still show variation.
Plot is unreadable	Reduce unnecessary features, choose meaningful axes, label units, increase figure size and avoid plotting every variable at once.
Text vocabulary is empty	Inspect cleaning, language assumptions, tokenization and <code>min_df</code> . Preserve raw and intermediate text for debugging.
K-Means clusters look wrong	Check scaling, <code>k</code> selection, initialization and whether the data actually has centroid-shaped clusters.
DBSCAN marks everything as noise	Inspect <code>eps</code> relative to the scaled feature space and reduce or increase it systematically rather than guessing.
SVM behaves poorly	Check scaling first, then examine <code>C</code> and <code>gamma</code> using validation evidence.
KNN behaves poorly	Check scaling, <code>k</code> , distance metric and irrelevant features.
Neural network does not converge	Check feature scaling, learning rate, <code>max_iter</code> and architecture size.

8. Final Project Readiness Boundary

Class 25 is the last instructional class in this manual. It prepares the student for the text-data research project but does not perform the final evaluation. The supplied course manual states that Classes 26–30 are reserved exclusively for final project evaluation and that no new project-development technique is assigned in those classes. [filecite](#)[turn0file0](#)[L111-L112](#)

What students should have ready after Class 25

- A reproducible notebook workflow.
- A clearly defined text-data problem and dataset source.
- Documented text cleaning and EDA procedures.
- A leakage-safe TF-IDF or comparable vectorization workflow.
- A simple baseline model.
- At least one additional candidate algorithm ready for experimentation.
- A model comparison structure with appropriate evaluation metrics.
- Saved figures and tables that can be reused in the research report.
- A record of preprocessing decisions, hyperparameters and experiment outcomes.
- A clear list of limitations, possible bias/fairness concerns and societal impact questions to investigate.

Project constraints carried forward from the supplied manual

The supplied project requirements specify that the project remains text-data focused; classification projects should use at least 1,000 labeled text samples unless an alternative research design is explicitly approved; at least two machine-learning algorithms and a simple baseline are required; appropriate metrics should be reported; binary classification should include a confusion matrix and ROC/AUC where appropriate; bias/fairness should be analyzed where meaningful; societal and environmental impact should be discussed; and the final report is specified as a 4–6 page IEEE-style paper with a private GitHub repository and a five-minute presentation. [filecite](#)[turn2file4](#)[L363-L385](#)

Important boundary: This manual deliberately stops at readiness. The final five evaluation classes are not reproduced here because they are assessment-only in the supplied course design.

Appendix A. Recommended Experiment Log

Field	Student Entry
Experiment ID	
Date	
Notebook	
Dataset / Version	
Random Seed	
Train/Test Split	
Preprocessing	
Model	
Hyperparameters	
Metric(s)	
Result	
Figure/Table Saved As	
Interpretation	
Limitation	

Next Experiment	
-----------------	--

Appendix B. Model Comparison Table Template

Model	Preprocessing	Accuracy	Precision	Recall	F1	ROC-AUC	Notes
Baseline							
Model 1							
Model 2							
Model 3							

Appendix C. Student Self-Assessment Before Leaving the Lab

- I can explain the algorithm without opening the code.
- I can explain why the chosen preprocessing is appropriate.
- I can identify the target variable and distinguish features from labels.
- I can identify possible sources of leakage.
- I can interpret the main metric rather than only reporting its value.
- I can explain one failure case or limitation.
- I can reproduce the experiment from a clean kernel.
- I can modify one parameter and predict what effect it may have before running the experiment.
- I can defend why one model is preferable to another using evidence.
- I can convert the experiment into a concise research-style result.

End of Classes 1–25 Guided Laboratory Manual. Classes 26–30 are intentionally excluded because the supplied course design reserves them for final project evaluation.